

1. Actor-Critic 算法简介

Actor-Critic 从名字上看包括两部分，演员(Actor)和评价者(Critic)。其中 Actor 使用我们上一节讲到的策略函数，负责生成动作(Action)并和环境交互。而 Critic 使用我们之前讲到的价值函数，负责评估 Actor 的表现，并指导 Actor 下一阶段的动作。

回想我们上一篇的策略梯度，策略函数就是我们的 Actor，但是那里是没有 Critic 的，我们当时使用了蒙特卡罗法来计算每一步的价值部分替代了 Critic 的功能，但是场景比较受限。因此现在我们使用类似 DQN 中用的价值函数来替代蒙特卡罗法，作为一个比较通用的 Critic。

也就是说在 Actor-Critic 算法中，我们需要做两组近似，第一组是策略函数的近似：

$$\pi_{\theta}(s,a)=P(a|s,\theta)\approx\pi(a|s)$$

第二组是价值函数的近似，对于状态价值和动作价值函数分别是：

$$\hat{V}(s,w)\approx v\pi(s)$$

$$\hat{Q}(s,a,w)\approx q\pi(s,a)$$

对于我们上一节讲到的蒙特卡罗策略梯度 reinforce 算法，我们需要进行改造才能变成 Actor-Critic 算法。

首先，在蒙特卡罗策略梯度 reinforce 算法中，我们的策略的参数更新公式是：

$$\theta = \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) v_t$$

梯度更新部分中， $\nabla_{\theta} \log \pi_{\theta}(s_t, a_t)$ 是我们的分值函数，不用动，要变成 Actor 的话改动的是 v_t ，这块不能再使用蒙特卡罗法来得到，而应该从 Critic 得到。

而对于 Critic 来说，这块是新的，不过我们完全可以参考之前 DQN 的做法，即用一个 Q 网络来做为 Critic，这个 Q 网络的输入可以是状态，而输出是每个动作的价值或者最优动作的价值。

现在我们汇总来说，就是 Critic 通过 Q 网络计算状态的最优价值 v_t ，而 Actor 利用 v_t 这个最优价值迭代更新策略函数的参数 θ ，进而选择动作，并得到反馈和新的状态，Critic 使用反馈和新的状态更新 Q 网络参数 w ，在后面 Critic 会使用新的网络参数 w 来帮 Actor 计算状态的最优价值 v_t 。

2. Actor-Critic 算法可选形式

在上一节我们已经对 Actor-Critic 算法的流程做了一个初步的总结，不过有一个可以注意的点就是，我们对于 Critic 评估的点选择是和上一篇策略梯度一样的状态价值 v_t ，实际上，我们还可以选择很多其他的指标来做为 Critic 的评估点。而目前可以使

用的 Actor-Critic 评估点主要有：

a) 基于状态价值：这是我们上一节使用的评估点，这样 Actor 的策略函数参数更新的法公式是：

$$\theta = \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) V(s, w)$$

b) 基于动作价值：在 DQN 中，我们一般使用的都是动作价值函数 Q 来做价值评估，这样 Actor 的策略函数参数更新的法公式是：

$$\theta = \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) Q(s, a, w)$$

c) 基于 TD 误差：TD 误差的表达式是 $\delta(t) = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ 或者 $\delta(t) = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$ ，这样 Actor 的策略函数参数更新的法公式是：

$$\theta = \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) \delta(t)$$

d) 基于优势函数 [Duelling DQN](#) 中，我们讲到过优势函数 A 的定义：

$A(S, A, w, \beta) = Q(S, A, w, \alpha, \beta) - V(S, w, \alpha)$ ，即动作价值函数和状态价值函数的差值。这样 Actor 的策略函数参数更新的法公式是：

$$A(S, A, w, \beta) = Q(S, A, w, \alpha, \beta) - V(S, w, \alpha)$$

e) 基于 TD(λ) 误差：一般都是基于后向 TD(λ) 误差，是 TD 误差和效用迹 E 的乘积。这样 Actor 的策略函数参数更新的法公式是：

$$A(S, A, w, \beta) = Q(S, A, w, \alpha, \beta) - V(S, w, \alpha)$$

对于 Critic 本身的模型参数 w，一般都是使用均方误差损失函数来做迭代更新，类似之前 DQN 系列中所讲的迭代方法。如果我们使用的是最简单的线性 Q 函数，比如 $Q(s, a, w) = \phi(s, a)^T w$ 则 Critic 本身的模型参数 w 的更新公式可以表示为：

$$\begin{aligned} \delta &= R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \\ w &= w + \beta \delta \phi(s, a) \end{aligned}$$

通过对均方误差损失函数求导可以很容易的得到上式。当然实际应用中，我们一般不使用线性 Q 函数，而使用神经网络表示状态和 Q 值的关系。

3. Actor-Critic 算法流程

这里给一个 Actor-Critic 算法的流程总结，评估点基于 TD 误差，Critic 使用神经网络来计算 TD 误差并更新网络参数，Actor 也使用神经网络来更新网络参数

算法输入：迭代轮数 T，状态特征维度 n，动作集 A，步长 α, β ，衰减因子 γ ，

探索率 ϵ , Critic 网络结构和 Actor 网络结构。

输出: Actor 网络参数 θ , Critic 网络参数 w

1. 随机初始化所有的状态和动作对应的价值 Q .

2. for i from 1 to T , 进行迭代。

a) 初始化 S 为当前状态序列的第一个状态, 拿到其特征向量 $\phi(S)$

b) 在 Actor 网络中使用 $\phi(S)$ 作为输入, 输出动作 A , 基于动作 A 得到新的状态 S' , 反馈 R 。

c) 在 Critic 网络中分别使用 $\phi(S)$, $\phi(S')$ 作为输入, 得到 Q 值输出 $V(S)$, $V(S')$

d) 计算 TD 误差 $\delta = R + \gamma V(S') - V(S)$

e) 使用均方差损失函数 $\sum (R + \gamma V(S') - V(S, w))^2$ 作 Critic 网络参数 w 的梯度更新

f) 更新 Actor 网络参数 θ :

$$\theta = \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(S_t, A) \delta$$

对于 Actor 的分值函数 $\nabla_{\theta} \log \pi_{\theta}(S_t, A)$, 可以选择 softmax 或者高斯分值函数。

上述 Actor-Critic 算法已经是一个很好的算法框架, 但是离实际应用还比较远。主要原因是这里有两个神经网络, 都需要梯度更新, 而且互相依赖。但是了解这个算法过程后, 其他基于 Actor-Critic 的算法就好理解了。